

# Today's Agenda:-

Starting 9:05

1. Intro To Number System
2. Binary To Decimal & Decimal To Binary
3. Addition of Binary Numbers.
4. Bitwise Operations.
5. Negative Numbers
6. Ranges
7. Importance of Constraints.

# Decimal Number System

↓  
10

→ Base 10 (Digits 0 to 9)

→ One we use in everyday lives.

→ Every digit represents a certain power of 10.

Ex:-

$$342 = 300 + 40 + 2 = 3 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$$

$$2563 = 2 \times 10^3 + 5 \times 10^2 + 6 \times 10^1 + 3 \times 10^0$$

# Binary Number System

↓  
2'

→ Base 2

→ Only 2 digits (0 & 1)

→ Used in electronic systems

Ex:-

$$\begin{array}{ccc} 2 & 1 & 0 \\ 1 & 1 & 0 \end{array} = 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$
$$= 4 + 2 + 0 = 6$$

$$\begin{array}{cccc} 3 & 2 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{array} = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$
$$= 8 + 0 + 2 + 1 = 11$$

# Binary To Decimal Conversion

Ex:-

3 2 1 0

1 1 0 1

$$\begin{aligned} & 1 \times 2^0 = 1 \\ & 0 \times 2^1 = 0 \\ & 1 \times 2^2 = 4 \\ & 1 \times 2^3 = 8 \end{aligned}$$

13

Ex:-

4 3 2 1 0

1 0 1 0 1

$$\begin{aligned} & 1 \times 2^0 = 1 \\ & 0 \times 2^1 = 0 \\ & 1 \times 2^2 = 4 \\ & 0 \times 2^3 = 0 \\ & 1 \times 2^4 = 16 \end{aligned}$$

21

Quiz 1

6 5 4 3 2 1 0

1 0 1 1 0 1 0

$$\begin{aligned} & 0 \times 2^0 = 0 \\ & 1 \times 2^1 = 2 \\ & 0 \times 2^2 = 0 \\ & 1 \times 2^3 = 8 \\ & 1 \times 2^4 = 16 \\ & 0 \times 2^5 = 0 \\ & 1 \times 2^6 = 64 \end{aligned}$$

90

# Decimal To Binary Conversion

→ Using long division method

Ex :-

$$(20)_{10} = (?.)_{2}$$

Quotients →

|   |    |   |
|---|----|---|
| 2 | 20 | 0 |
| 2 | 10 | 0 |
| 2 | 5  | 1 |
| 2 | 2  | 0 |
| 2 | 1  | 1 |
|   | 0  |   |
|   |    |   |

Remainders ↑

(10100)<sub>2</sub>

Quiz 2

$$(45)_{10} = (?.)_{2}$$

|   |    |   |
|---|----|---|
| 2 | 45 | 1 |
| 2 | 22 | 0 |
| 2 | 11 | 1 |
| 2 | 5  | 1 |
| 2 | 2  | 0 |
| 2 | 1  | 1 |
|   | 0  |   |

(101101)<sub>2</sub>

# Addition of Decimal Numbers

$$\begin{array}{r}
 0168 \\
 + 253 \\
 \hline
 621
 \end{array}$$

| digit        | Carry   |
|--------------|---------|
| $11 \div 10$ | $11/10$ |
| $12 \div 10$ | $12/10$ |
| $6 \div 10$  | $6/10$  |

# Addition of Binary Numbers

Ex:-

$$\begin{array}{r}
 011101 \\
 + 1101 \\
 \hline
 100010
 \end{array}$$

| digit      | Carry |
|------------|-------|
| $2 \div 2$ | $2/2$ |
| $1 \div 2$ | $1/2$ |
| $2 \div 2$ | $2/2$ |
| $2 \div 2$ | $2/2$ |
| $2 \div 2$ | $2/2$ |
| $1 \div 2$ | $1/2$ |

Q:-3

$$\begin{array}{r}
 00110 \\
 + 00111 \\
 \hline
 11101
 \end{array}$$

| digit      | Carry |
|------------|-------|
| $1 \div 2$ | $1/2$ |
| $2 \div 2$ | $2/2$ |
| $3 \div 2$ | $3/2$ |
| $1 \div 2$ | $1/2$ |
| $1 \div 2$ | $1/2$ |

# Bitwise Operators

&, |, ^, ~, >>, <<

→ Operators used to perform operations on individual bits of binary numbers.

→ Used to manipulate binary data

→ 0 → false / unset  
1 → true / set

## AND (&)

→ Performs Logical AND

→ Return true if both bits are set, else false.

|   |   |   | <u>Result</u> |
|---|---|---|---------------|
| 0 | & | 0 | 0             |
| 0 | & | 1 | 0             |
| 1 | & | 0 | 0             |
| 1 | & | 1 | 1             |

## OR (|)

- Performs Logical OR
- Return true if either of bits are set, else false.

|   |   | <u>Result</u> |
|---|---|---------------|
| 0 | 0 | 0             |
| 0 | 1 | 1             |
| 1 | 0 | 1             |
| 1 | 1 | 1             |

## XOR (^)

- Perform exclusive OR.
- Return true if both bits are different, else false.

|   |   | <u>Result</u> |
|---|---|---------------|
| 0 | 0 | 0             |
| 0 | 1 | 1             |
| 1 | 0 | 1             |
| 1 | 1 | 0             |

# NOT (~) !

- Operates on a single bit
- Toggles it.

|          | <u>Result</u> |
|----------|---------------|
| $\sim 0$ | 1             |
| $\sim 1$ | 0             |

## Examples

①      5   \*   6

          4

←

$$\begin{array}{r} 101 \\ * 110 \\ \hline 100 \end{array}$$

②      20   \*   45

          9

$$\begin{array}{r} 010100 \\ * 101101 \\ \hline 000100 \end{array}$$

③  $92 \wedge 154$

198

$$\begin{array}{r}
 01011100 \\
 \wedge 10011010 \\
 \hline
 11000110 \\
 128 \quad 64 \quad \quad \quad 4 \quad 2
 \end{array}$$

Quiz 4

$20 \wedge 45$

57

$$\begin{array}{r}
 010100 \\
 \wedge 101101 \\
 \hline
 111001 \\
 \hline
 32 \quad 16 \quad 8 \quad \quad \quad 1
 \end{array}$$

2, 4, 8, 16, 32, 64, 128, 256, 512, 1024.

# Binary Representation of Negative Numbers

ex)- Represent  $(-5)$  in 8 bit signed format.

Three Step Process:-  $\nearrow$  2's complement of 5:

1. Represent the abs. value of the no. in binary.

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |

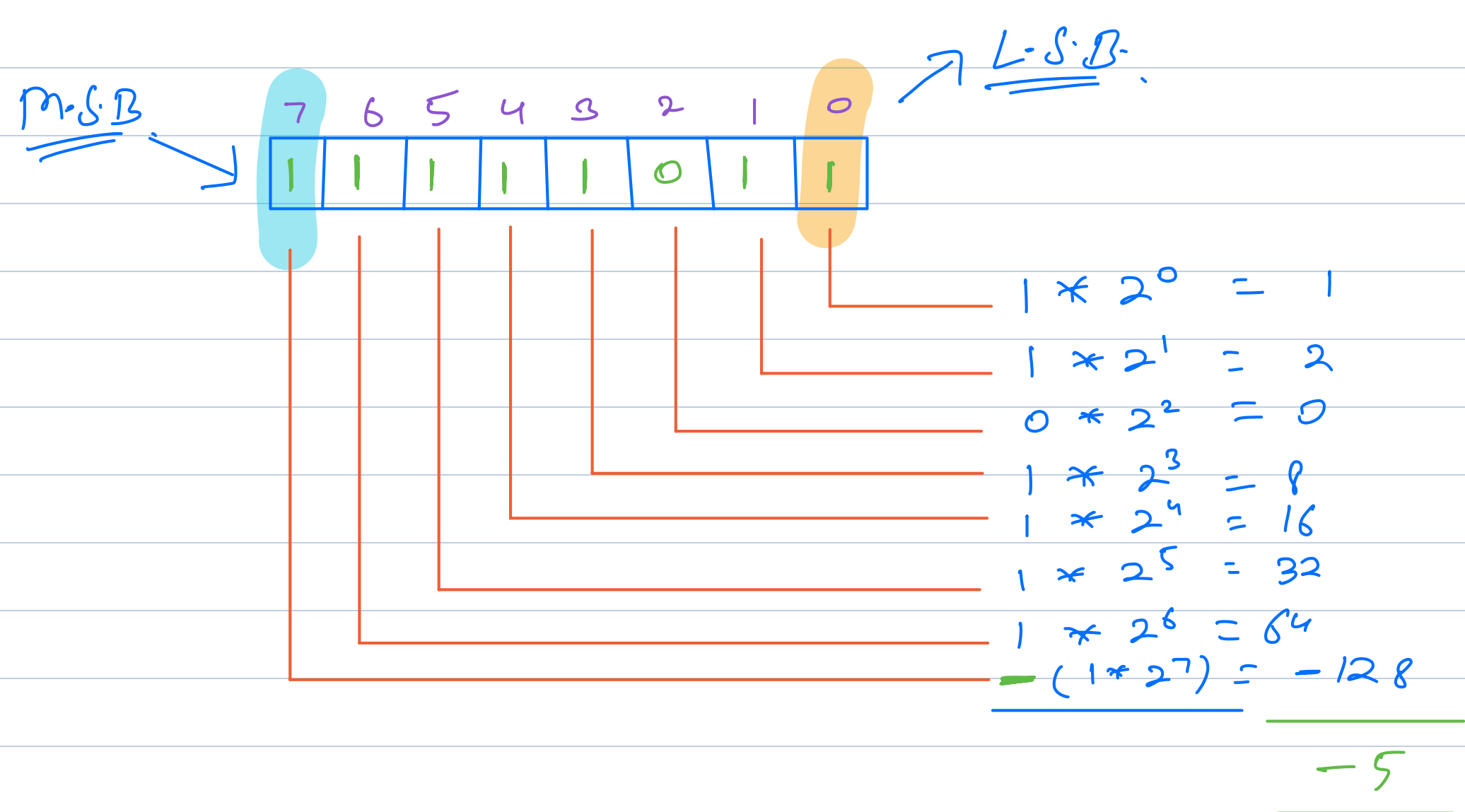
② Apply negation ( $\sim$ ) on the result of Step 1.

| 7        | 6        | 5        | 4        | 3        | 2        | 1        | 0        |
|----------|----------|----------|----------|----------|----------|----------|----------|
| $\sim$ 0 | $\sim$ 0 | $\sim$ 0 | $\sim$ 0 | $\sim$ 0 | $\sim$ 1 | $\sim$ 0 | $\sim$ 1 |

| 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |
|-----|-----|-----|-----|-----|-----|-----|-----|
| = 1 | = 1 | = 1 | = 1 | = 1 | = 0 | = 1 | = 0 |

③ Add 1 to the result of Step 2.

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 |
|   |   |   |   |   |   | + | 1 |
| = | 1 | 1 | 1 | 1 | 1 | 0 | 1 |

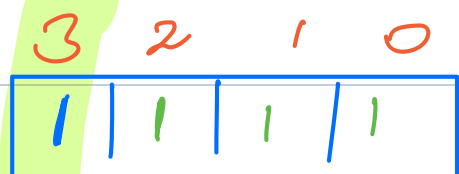


The M.S.B. is always going to determine whether the no. is positive or negative.

If MSB is 1  $\rightarrow$  No is always negative

else

$\rightarrow$  No is always positive.



$$-1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

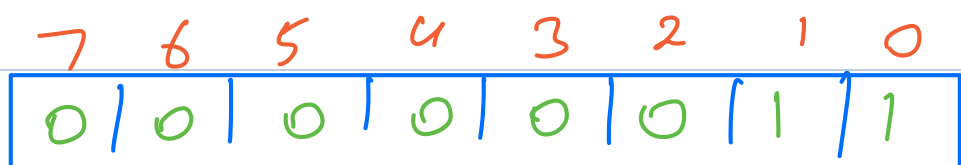
$$= -8 + 7$$

$$= -1$$

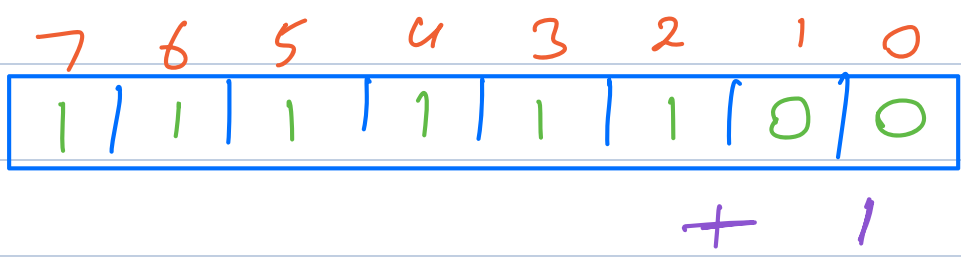
Quiz 5

-3 in 8-bit signed integer format.

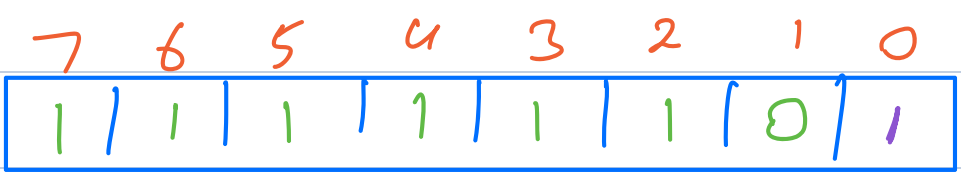
①



②



③



10:16,

# Range of Data Types

|   |   |   |   |
|---|---|---|---|
| 3 | 2 | 1 | 0 |
| 1 | 0 | 0 | 0 |

$$-2^3$$

$$\underline{-8 \leq A \leq 7}$$

|   |   |   |   |
|---|---|---|---|
| 3 | 2 | 1 | 0 |
| 0 | 1 | 1 | 1 |

$$\begin{aligned} &= 2^0 + 2^1 + 2^2 \\ &= 2^0 \frac{(2^3 - 1)}{2 - 1} \\ &= 2^3 - 1 \\ &= 7 \end{aligned}$$

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

$$-2^7$$

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

$$\begin{aligned} &= 2^0 + 2^1 + 2^2 + \dots + 2^6 \\ &= 2^7 - 1 \end{aligned}$$

$$\underline{-128 \leq A \leq 127}$$

Generalisation for a N bit data type.

$$\boxed{-2^{N-1} \leq A \leq +2^{N-1} - 1}$$

For an int data type;  
 $N = 32$

$$\underline{-2^{31} \leq \text{int} \leq 2^{31} - 1}$$

$$\underline{\sim -2 \times 10^9 \leq \text{int} \leq +2 \times 10^9}$$

Similarly for a long data type.  
 $N = 64$ .

$$-2^{63} \leq \text{long} \leq +2^{63} - 1$$

$$\underline{\sim -9 \times 10^{18} \leq \text{long} \leq +9 \times 10^{18}}$$

# Importance of Constraints

int a =  $10^5$   
int b =  $10^6$

Trying to compute  $a * b$ :

1. int c = a \* b; X;

2. long c = a \* b;

ALU.

3. long c = long(a) \* b; ✓  
= 1L \* a \* b; ✓

Q:-  $1 \leq N \leq 10^5$   
 $1 \leq A[i] \leq 10^6$

Given an array of size  $N$ , Cal. the sum of array elements.

```
long:  
int sum = 0;  
for (i  $\rightarrow$  0 to  $N-1$ ) {  
    | sum += arr[i];  
}  
3  
return sum;
```

---

```
int x = -6;
```